

CM2 - Représentation d'informations dans un ordinateur

Ammar Mian

Associate professor, LISTIC, Université Savoie Mont Blanc
POLYTECH Annecy-Chambéry

Plan du cours

1. Notion de Codage
2. Entiers non signés
3. Entiers signés
4. Nombres à virgule flottante
5. Représentation des caractères
6. Exercices de synthèse

Objectifs de ce cours

À l'issue de ce CM, vous serez capable de :

- Comprendre le principe du codage binaire
- Convertir des nombres entre différentes bases (2, 8, 10, 16)
- Représenter des entiers signés avec le complément à 2
- Comprendre la représentation des nombres flottants (IEEE 754)
- Connaître les principaux codages de caractères (ASCII, Unicode)
- Identifier les limites et pièges de ces représentations

Important

Ce cours nécessite un travail personnel pour approfondir les notions.

1. Notion de Codage

2. Entiers non signés

3. Entiers signés

4. Nombres à virgule flottante

5. Représentation des caractères

6. Exercices de synthèse

Notion de codage

Définition

Un **codage** est un processus **bijectif** qui permet de passer d'une représentation d'informations à une autre.

En informatique :

- Toute information est représentée par une suite de 0 ou 1
- Un **bit** (binary digit) : $x \in \{0, 1\}$
- Un bit prend exactement deux valeurs différentes

Exemple simple :

- Off $\rightarrow 0$
- On $\rightarrow 1$

Fonction de codage :

$$f : \{\text{Off}, \text{On}\} \rightarrow \{0, 1\}$$

Fonction de décodage :

$$f^{-1} : \{0, 1\} \rightarrow \{\text{Off}, \text{On}\}$$



Combien d'informations peut-on coder ? I

Avec n bits, on peut représenter 2^n informations différentes :

- 1 bit $\rightarrow 2^1 = 2$ informations
- 2 bits $\rightarrow 2^2 = 4$ informations
- 3 bits $\rightarrow 2^3 = 8$ informations
- $\vdots$
- n bits $\rightarrow 2^n$ informations

Exercice : Codage des plaques d'immatriculation

Depuis 2009, la France utilise un nouveau système de numérotation :

Format : AB-123-CD

(2 lettres - 3 chiffres - 2 lettres)

Contraintes :

- Lettres I, O, U non utilisées (confusion avec 1, 0, V)
- Couple SS interdit à gauche et à droite
- Couple WW interdit à droite

Questions (2-3 min de réflexion)

1. Combien de véhicules peut-on immatriculer avec ce système ?
2. Combien de bits faudrait-il pour coder chaque véhicule ?
3. Combien de bits si on code séparément lettres et chiffres ?
4. Sachant qu'il y avait 37M de véhicules en 2009 et 3M de nouvelles immatriculations/an, quelle est la durée de vie de ce système ?

Correction de l'exercice

1. Nombre de véhicules possibles :

Correction de l'exercice

1. Nombre de véhicules possibles :

- Lettres disponibles : $26 - 3 = 23$ lettres
- Partie gauche (2 lettres) : $23^2 - 1 = 528$ (retirer SS)
- Partie centrale (3 chiffres) : $10^3 = 1000$
- Partie droite (2 lettres) : $23^2 - 2 = 527$ (retirer SS et WW)
- **Total** : $528 \times 1000 \times 527 = 278\,256\,000$ plaques

Correction de l'exercice

1. Nombre de véhicules possibles :

- Lettres disponibles : $26 - 3 = 23$ lettres
- Partie gauche (2 lettres) : $23^2 - 1 = 528$ (retirer SS)
- Partie centrale (3 chiffres) : $10^3 = 1000$
- Partie droite (2 lettres) : $23^2 - 2 = 527$ (retirer SS et WW)
- **Total** : $528 \times 1000 \times 527 = 278\,256\,000$ plaques

2. Bits pour coder chaque véhicule :

$$\lceil \log_2(278\,256\,000) \rceil = \lceil 27.73 \rceil = 28 \text{ bits}$$

Correction de l'exercice

1. Nombre de véhicules possibles :

- Lettres disponibles : $26 - 3 = 23$ lettres
- Partie gauche (2 lettres) : $23^2 - 1 = 528$ (retirer SS)
- Partie centrale (3 chiffres) : $10^3 = 1000$
- Partie droite (2 lettres) : $23^2 - 2 = 527$ (retirer SS et WW)
- **Total** : $528 \times 1000 \times 527 = 278\,256\,000$ plaques

2. Bits pour coder chaque véhicule :

$$\lceil \log_2(278\,256\,000) \rceil = \lceil 27.73 \rceil = 28 \text{ bits}$$

3. Codage séparé :

- Gauche : $\lceil \log_2(528) \rceil = 10$ bits
- Centre : $\lceil \log_2(1000) \rceil = 10$ bits
- Droite : $\lceil \log_2(527) \rceil = 10$ bits
- **Total** : **30 bits** (moins optimal !)

Correction de l'exercice

1. Nombre de véhicules possibles :

- Lettres disponibles : $26 - 3 = 23$ lettres
- Partie gauche (2 lettres) : $23^2 - 1 = 528$ (retirer SS)
- Partie centrale (3 chiffres) : $10^3 = 1000$
- Partie droite (2 lettres) : $23^2 - 2 = 527$ (retirer SS et WW)
- **Total** : $528 \times 1000 \times 527 = 278\,256\,000$ plaques

2. Bits pour coder chaque véhicule :

$$\lceil \log_2(278\,256\,000) \rceil = \lceil 27.73 \rceil = 28 \text{ bits}$$

3. Codage séparé :

- Gauche : $\lceil \log_2(528) \rceil = 10$ bits
- Centre : $\lceil \log_2(1000) \rceil = 10$ bits
- Droite : $\lceil \log_2(527) \rceil = 10$ bits
- **Total** : **30 bits** (moins optimal !)

4. Durée de vie :

$$\frac{278\,256\,000 - 37\,000\,000}{3\,000\,000} \approx 80 \text{ ans}$$

→ Système viable jusqu'à environ **2089**

1. Notion de Codage
2. Entiers non signés
3. Entiers signés
4. Nombres à virgule flottante
5. Représentation des caractères
6. Exercices de synthèse

Représentation des nombres en différentes bases

Un nombre est une suite de chiffres lus de gauche à droite. Pour une base B , chaque chiffre appartient à un ensemble de B symboles.

Bases courantes :

Base	Nom	Symboles
Base 2	Binaire	$\{0, 1\}$
Base 8	Octale	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Base 10	Décimale	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Base 16	Hexadécimale	$\{0-9, A, B, C, D, E, F\}$

Représentation des nombres en différentes bases

Un nombre est une suite de chiffres lus de gauche à droite. Pour une base B , chaque chiffre appartient à un ensemble de B symboles.

Bases courantes :

Base	Nom	Symboles
Base 2	Binaire	$\{0, 1\}$
Base 8	Octale	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Base 10	Décimale	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Base 16	Hexadécimale	$\{0-9, A, B, C, D, E, F\}$

Notation :

- N_B : nombre N exprimé en base B
- N (sans indice) : nombre en base 10
- $(N_A)_B$: conversion de N de la base A vers la base B

Représentation des nombres en différentes bases

Un nombre est une suite de chiffres lus de gauche à droite. Pour une base B , chaque chiffre appartient à un ensemble de B symboles.

Bases courantes :

Base	Nom	Symboles
Base 2	Binaire	$\{0, 1\}$
Base 8	Octale	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Base 10	Décimale	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Base 16	Hexadécimale	$\{0-9, A, B, C, D, E, F\}$

Notation :

- N_B : nombre N exprimé en base B
- N (sans indice) : nombre en base 10
- $(N_A)_B$: conversion de N de la base A vers la base B

Exemples :

- $(9_{10})_2 = 1001_2$
- $(11_2)_{10} = 3$
- $(111101_2)_{16} = 1D_{16}$

Principe de conversion vers la base 10

Soit N_B un nombre en base B sur n chiffres : $N_B = x_{n-1} \dots x_2 x_1 x_0$

Formule de conversion

$$(N_B)_{10} = p_{n-1} \cdot B^{n-1} + \dots + p_2 \cdot B^2 + p_1 \cdot B^1 + p_0 \cdot B^0$$

où p_i est le **poids** du chiffre x_i en base 10.

Poids d'un chiffre :

- Pour les bases ≤ 10 : le poids = le chiffre lui-même
- Pour la base 16 : A=10, B=11, C=12, D=13, E=14, F=15

Principe de conversion vers la base 10

Soit N_B un nombre en base B sur n chiffres : $N_B = x_{n-1} \dots x_2 x_1 x_0$

Formule de conversion

$$(N_B)_{10} = p_{n-1} \cdot B^{n-1} + \dots + p_2 \cdot B^2 + p_1 \cdot B^1 + p_0 \cdot B^0$$

où p_i est le **poids** du chiffre x_i en base 10.

Poids d'un chiffre :

- Pour les bases ≤ 10 : le poids = le chiffre lui-même
- Pour la base 16 : A=10, B=11, C=12, D=13, E=14, F=15

Exemples :

1) $(1011)_2$ en base 10 :

$$= 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11$$

2) $(2A)_{16}$ en base 10 :

$$= 2 \cdot 16^1 + 10 \cdot 16^0 = 32 + 10 = 42$$

Exercice : Conversions vers la base 10

a) $T = 323_8$ b) $U = 323_{16}$ c) $V = 1AF_{16}$ d) $W = 323_3$ e) $X = 111011_2$

Exercice : Conversions vers la base 10

a) $T = 323_8$ b) $U = 323_{16}$ c) $V = 1AF_{16}$ d) $W = 323_3$ e) $X = 111011_2$

Résolution ensemble

$$\text{a) } T = 323_8 = 3 \cdot 8^2 + 2 \cdot 8^1 + 3 \cdot 8^0 = 192 + 16 + 3 = 211$$

$$\text{b) } U = 323_{16} = 3 \cdot 16^2 + 2 \cdot 16^1 + 3 \cdot 16^0 = 768 + 32 + 3 = 803$$

$$\text{c) } V = 1AF_{16} = 1 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0 = 256 + 160 + 15 = 431$$

Exercice : Conversions vers la base 10

a) $T = 323_8$ b) $U = 323_{16}$ c) $V = 1AF_{16}$ d) $W = 323_3$ e) $X = 111011_2$

Résolution ensemble

$$\text{a) } T = 323_8 = 3 \cdot 8^2 + 2 \cdot 8^1 + 3 \cdot 8^0 = 192 + 16 + 3 = 211$$

$$\text{b) } U = 323_{16} = 3 \cdot 16^2 + 2 \cdot 16^1 + 3 \cdot 16^0 = 768 + 32 + 3 = 803$$

$$\text{c) } V = 1AF_{16} = 1 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0 = 256 + 160 + 15 = 431$$

d) ATTENTION : Piège !

$W = 323_3$ est **INVALIDE** ! En base 3, seuls les symboles $\{0, 1, 2\}$ sont autorisés. Le chiffre "3" n'existe pas en base 3.

$$\text{e) } X = 111011_2 = 32 + 16 + 8 + 2 + 1 = 59$$

Méthode de conversion Décimal $\leftrightarrow$ BinaireDécimal $\rightarrow$ Binaire

Divisions successives par 2

Exemple : $125_{10} \rightarrow ?_2$

$$125 \div 2 = 62 \text{ reste } 1$$

$$62 \div 2 = 31 \text{ reste } 0$$

$$31 \div 2 = 15 \text{ reste } 1$$

$$15 \div 2 = 7 \text{ reste } 1$$

$$7 \div 2 = 3 \text{ reste } 1$$

$$3 \div 2 = 1 \text{ reste } 1$$

$$1 \div 2 = 0 \text{ reste } 1$$

Résultat : 1111101_2 Lecture
bas $\rightarrow$ hautBinaire $\rightarrow$ Décimal

Somme des puissances de 2

Exemple : $1111101_2 \rightarrow ?_{10}$

Position:	0	1	2	3	4	5	6
Bit:	1	1	1	1	1	0	1
Puissance:	64	32	16	8	4	2	1
	↓	↓	↓	↓	↓		↓
	64	32	16	8	4	0	1
	+ + + + +						
	= 125						
	₁₀						

Exercice : Convertir 125 en différentes bases

Convertir le nombre 125 en : a) Base 2, b) Base 8, c) Base 16

Exercice : Convertir 125 en différentes bases

Convertir le nombre 125 en : a) Base 2, b) Base 8, c) Base 16

a) Conversion en base 2 : (cf slide précédent) $125_{10} = 1111101_2$

b) Conversion en base 8 :

Exercice : Convertir 125 en différentes bases

Convertir le nombre 125 en : a) Base 2, b) Base 8, c) Base 16

a) Conversion en base 2 : (cf slide précédent) $125_{10} = 1111101_2$

b) Conversion en base 8 :

- $125 \div 8 = 15$ reste 5
- $15 \div 8 = 1$ reste 7
- $1 \div 8 = 0$ reste 1
- Lecture de bas en haut : 175_8

c) Conversion en base 16 :

Exercice : Convertir 125 en différentes bases

Convertir le nombre 125 en : a) Base 2, b) Base 8, c) Base 16

a) Conversion en base 2 : (cf slide précédent) $125_{10} = 1111101_2$

b) Conversion en base 8 :

- $125 \div 8 = 15$ reste 5
- $15 \div 8 = 1$ reste 7
- $1 \div 8 = 0$ reste 1
- Lecture de bas en haut : 175_8

c) Conversion en base 16 :

- $125 \div 16 = 7$ reste 13 (D)
- $7 \div 16 = 0$ reste 7
- Lecture de bas en haut : $7D_{16}$

Vérification

On retrouve bien 125 depuis chaque base !

- $175_8 = 1 \cdot 64 + 7 \cdot 8 + 5 = 125$
- $7D_{16} = 7 \cdot 16 + 13 = 125$

Passerelle rapide Binaire $\leftrightarrow$ Hexadécimal

Binaire $\rightarrow$ Hexa

Grouper par 4 bits (nibble)

Exemple : $11111101_2 = ?_{16}$

1111	1101
------	------

↓	↓
F	D

FD_{16}

Hexa $\rightarrow$ Binaire

Développer chaque symbole en 4 bits

Exemple : $7D_{16} = ?_2$

7	D
↓	↓
0111	1101

01111101_2

Représentation sur un nombre de bits fixe I

Notation : ${}^P(N_A)_B$

Conversion de N de la base A vers la base B sur **exactement** P chiffres.

Exemples :

- ${}^8(9_{10})_2 = 00001001_2$ (9 en binaire sur 8 bits)
- ${}^4(11_2)_{10} = 0003$ (11 binaire = 3, sur 4 chiffres)

Représentation sur un nombre de bits fixe II

En programmation

Les entiers sont codés sur un nombre de bits fixe :

Type	Bits	Plage (non signé)
char / uint8_t	8 bits	0 à 255
short / uint16_t	16 bits	0 à 65 535
int / uint32_t	32 bits	0 à 4 294 967 295
long / uint64_t	64 bits	0 à $2^{64} - 1$

Addition en binaire (nombres non signés)

Principe : Addition chiffre par chiffre, de droite à gauche, en propageant la retenue.

Notation : $+_B$ pour l'addition dans la base B

Exemples :

En base 10 :

$$\begin{array}{r} 81 \\ + 89 \\ \hline 170 \end{array}$$

En base 16 :

$$\begin{array}{r} 81 \\ + 89 \\ \hline 10A \end{array}$$

En base 2 :

$$\begin{array}{r} 00101 \\ + 01110 \\ \hline 10011 \end{array}$$

$$(5 + 14 = 19)$$

Détection de débordement

Sur un format de n bits : une retenue finale indique un **débordement** = résultat incorrect.

SANS débordement : $28 + 5$

```
Retenue:      11
              00011100 (28)
+ 00000101 (5)
-----
              00100001 (33)
```

Pas de retenue finale

→ Correct !

AVEC débordement : $128 + 129$

```
Retenue: 1
          10000000 (128)
+ 10000001 (129)
-----
1 00000001 (1)
  ^
```

Retenue finale = 1

→ **X DÉBOREMENT**

Résultat théorique : 257

Plage sur 8 bits : 0 à 255

→ 257 non représentable

Détection de débordement

Sur un format de n bits : une retenue finale indique un **débordement** = résultat incorrect.

SANS débordement : $28 + 5$

```
Retenue:      11
              00011100 (28)
+ 00000101 (5)
-----
              00100001 (33)
```

Pas de retenue finale

→ Correct !

AVEC débordement : $128 + 129$

```
Retenue: 1
          10000000 (128)
+ 10000001 (129)
-----
1 00000001 (1)
  ^
```

Retenue finale = 1

→ **X DÉBOREMENT**

Résultat théorique : 257

Plage sur 8 bits : 0 à 255

→ 257 non représentable

Sur les processeurs

Un indicateur **C** (Carry) est positionné quand il y a débordement.

En C, il faut gérer ces débordements soi-même !

Récapitulatif : Entiers non signés

Ce qu'on a vu :

- ✓ **Conversion Base $B \rightarrow$ Base 10 :**
Somme des poids $\times$ puissances de B
- ✓ **Conversion Base 10 $\rightarrow$ Base B :**
Divisions successives par B , lecture des restes
- ✓ **Astuce Binaire $\leftrightarrow$ Hexa :**
Groupement par nibbles (4 bits)
- ✓ **Addition binaire :**
Chiffre par chiffre avec propagation des retenues
- ✓ **Débordement :**
Retenue finale = indicateur $C =$ overflow

Limitations

- Format fixe $\rightarrow$ plage limitée
- Pas de nombres négatifs pour l'instant

Question : Comment représenter -5 ?

1. Notion de Codage

2. Entiers non signés

3. Entiers signés

4. Nombres à virgule flottante

5. Représentation des caractères

6. Exercices de synthèse

Comment représenter des nombres négatifs ? I

Approche naïve : Bit de signe + Valeur absolue

Idée : Utiliser 1 bit pour le signe, le reste pour la valeur.

Par exemple sur 4 bits : $[S] [x_2] [x_1] [x_0]$

- $S=0 \rightarrow$ positif
- $S=1 \rightarrow$ négatif

Exemple : $+3 = 0011_2$, $-3 = 1011_2$

Comment représenter des nombres négatifs ? II

Deux problèmes majeurs

X Problème 1 : Double représentation du zéro

- $+0 = 0000_2$
- $-0 = 1000_2$
- $\rightarrow$ Gaspillage d'un code

X Problème 2 : L'addition ne fonctionne pas !

Calcul : $(+3) + (-2)$ devrait donner $+1$

```

    0011  (+3)
+   1010  (-2)
-----
    1101   $\rightarrow$  décodage = -5   xxx
  
```

Code complément : une meilleure approche

Principe sur 4 bits (16 codes possibles) :

- 8 codes pour les positifs : 0 à +7 → Codage classique : ${}^4(N)_2$
- 8 codes pour les négatifs : -8 à -1 → Codage spécial pour que l'addition fonctionne

Équation fondamentale

$${}^P(N)_B + {}^P(-N)_B = {}^P(0)_B$$

Le code du négatif doit être tel que son addition avec le positif donne 0 (en ignorant la retenue sortante).

Exemple intuitif en base 10 sur 4 chiffres :

Pour trouver le code de -3 : $0003 + \text{????} = 0000$ (en gardant 4 chiffres)

Quelle valeur ajouter à 3 pour obtenir 0000 ?

$$0003 + 9997 = 10000 \rightarrow \text{les 4 derniers chiffres} = 0000$$

Donc : ${}^4(-3) = 9997$

Vérification : $0003 + 9997 = 1|0000$ (on garde les 4 derniers)

Définition mathématique du complément I

Complément d'un chiffre

Soit x_B un chiffre en base B .

Le complément de x_B , noté x_B^* , est défini par :

$$x_B +_B x_B^* = (B - 1)_B$$

Exemples :

- En base 2 : $1_2^* = 0_2$ car $1_2 +_2 0_2 = 1_2$, $0_2^* = 1_2$ car $0_2 +_2 1_2 = 1_2$
- En base 8 : $1_8^* = 6_8$ car $1_8 +_8 6_8 = 7_8$
- En base 16 : $1_{16}^* = E_{16}$ car $1_{16} +_{16} E_{16} = F_{16}$

Définition mathématique du complément II

Propriété : Le complément est involutif : $x_B^{**} = x_B$

Complément d'un nombre

Pour $^P(N)_B$ sur P chiffres, on applique le complément à chaque chiffre.

Exemples :

- $01011000_2^* = 10100111_2$ (on inverse chaque bit)
- $108^* = 891$ (en base 10 sur 3 chiffres)

Complément à 2 : la formule

Théorème

$${}^P(-N)_B = {}^P(N)_B^* +_B 1$$

C'est-à-dire : Code du négatif = Complément de chaque chiffre + 1

Pourquoi ça marche ?

On sait que : ${}^P(N)_B +_B {}^P(N)_B^* = B^P - 1$

Donc : $B^P = ({}^P(N)_B +_B {}^P(N)_B^*)_{10} + 1$

Or on cherche : ${}^P(-N)_B = {}^P(B^P - N)_B$

En remplaçant : ${}^P(-N)_B = {}^P(({}^P(N)_B^*)_{10} + 1)_B = {}^P(N)_B^* +_B 1$

Méthode pratique pour coder un négatif

1. Écrire le positif
2. Inverser tous les chiffres
3. Ajouter 1

Exemple : Coder -3 en base 2 sur 8 bits I

Étape 1 : Coder +3 en binaire sur 8 bits

$$^8(3)_2 = 00000011_2$$

Étape 2 : Inverser tous les bits (complément)

$$00000011_2^* = 11111100_2$$

Étape 3 : Ajouter 1

$$\begin{array}{r}
 11111100 \\
 + \quad \quad 1 \\
 - - - - - \\
 11111101
 \end{array}$$

Exemple : Coder -3 en base 2 sur 8 bits II

Vérification

$$00000011_2 + 11111101_2 = ?$$

Retenue: 11111111

$$\begin{array}{r}
 00000011 \quad (+3) \\
 + 11111101 \quad (-3) \\
 \hline
 1\ 00000000 \quad (0) \\
 \wedge
 \end{array}$$

On ignore la retenue sortante

Le résultat est bien 0 !

Exemple : Coder -3 en base 10 sur 4 chiffres

Étape 1 : Coder +3 en base 10 sur 4 chiffres

$${}^4(3) = 0003$$

Étape 2 : Complémenter chaque chiffre (en base 10, le complément de x est $9 - x$)

$$0003^* = 9996$$

- $0 \rightarrow 9$
- $3 \rightarrow 6$

Étape 3 : Ajouter 1

$$\begin{array}{r}
 9996 \\
 + \quad 1 \\
 \hline
 9997
 \end{array}$$

On retrouve bien le résultat intuitif du slide 18 !

Exercice : Calculer des codes en complément à 2

Questions :

- a) Code de 3 puis de -3 en base 2 sur 8 bits
- b) Code de 3 puis de -3 en base 4 sur 3 chiffres
- c) Code de -128 en base 2 sur 8 bits
- d) Code de -3 en base 16 sur 4 chiffres

Bonus :

- e) Si $X = {}^P(-N)_B$, que vaut $X^* + 1$?

Correction

a) Base 2 sur 8 bits : $+3 = 00000011_2$, $-3 = 11111101_2$

b) Base 4 sur 3 chiffres : $+3 = 003_4$

- Complément en base 4 : $0 \rightarrow 3$, $3 \rightarrow 0$
- $003_4^* = 330_4$
- $-3 = 330_4 + 1 = 331_4$

c) -128 en base 2 sur 8 bits :

- +128 ne tient pas sur 8 bits signés ! (max = 127)
- Mais -128 est représentable (min = -128)
- Astuce : $-128 = -2^7 = 10000000_2$

d) -3 en base 16 sur 4 chiffres :

- $+3 = 0003_{16}$, $0003_{16}^* = FFFC_{16}$, $-3 = FFFD_{16}$

e) Si $X = {}^P(-N)_B = {}^P(N)_B^* + 1$, alors :

$$X^* + 1 = ({}^P(N)_B^* + 1)^* + 1 = {}^P(N)_B^{**} + \dots = {}^P(N)_B$$

→ Le complément à 2 est involutif : $-(-N) = N$

Comment reconnaître un nombre négatif ? I

Propriété du code complément à 2

Le bit (ou chiffre) de poids fort indique le signe. En base 2 sur n bits :

- Bit de poids fort = 0 $\rightarrow$ nombre positif ou nul
- Bit de poids fort = 1 $\rightarrow$ nombre négatif

On l'appelle le "bit de signe".

Exemples sur 8 bits :

Code	Bit signe	Valeur
0 0000000	0	+0
0 0000011	0	+3
0 1111111	0	+127 (max)
1 0000000	1	-128 (min)
1 1111101	1	-3
1 1111111	1	-1

Comment reconnaître un nombre négatif ? II

Plage sur n bits

$$[-2^{n-1}, 2^{n-1} - 1]$$

Sur 8 bits : $[-128, +127]$ (256 valeurs, asymétrique)

Différence avec signe + valeur absolue

Le bit de poids fort n'est PAS simplement collé devant la valeur !

Exercice : Décodage en Complément à 2

Consigne

Convertissez les nombres binaires suivants (codés sur 8 bits en complément à 2) en décimal :

1. $A = 0101\ 1010_2$
2. $B = 1000\ 0001_2$
3. $C = 1111\ 1111_2$
4. $D = 0111\ 1111_2$

Exercice : Décodage en Complément à 2

Consigne

Convertissez les nombres binaires suivants (codés sur 8 bits en complément à 2) en décimal :

1. $A = 0101\ 1010_2$
2. $B = 1000\ 0001_2$
3. $C = 1111\ 1111_2$
4. $D = 0111\ 1111_2$

Méthode

- Si bit de poids fort = 0 $\rightarrow$ nombre positif $\rightarrow$ conversion classique
- Si bit de poids fort = 1 $\rightarrow$ nombre négatif $\rightarrow$ formule du complément à 2

Correction : Décodage en Complément à 2

- a) $A = 0101\ 1010_2 \rightarrow$ Bit de poids fort = 0 $\rightarrow$ positif

$$A = 64 + 16 + 8 + 2 = \mathbf{90}_{10}$$

Correction : Décodage en Complément à 2

- a) $A = 0101\ 1010_2 \rightarrow$ Bit de poids fort = 0 $\rightarrow$ positif

$$A = 64 + 16 + 8 + 2 = \mathbf{90}_{10}$$

- b) $B = 1000\ 0001_2 \rightarrow$ Bit de poids fort = 1 $\rightarrow$ négatif

$$B = -128 + 1 = \mathbf{-127}_{10}$$

Correction : Décodage en Complément à 2

- a) $A = 0101\ 1010_2 \rightarrow$ Bit de poids fort = 0 $\rightarrow$ positif

$$A = 64 + 16 + 8 + 2 = \mathbf{90}_{10}$$

- b) $B = 1000\ 0001_2 \rightarrow$ Bit de poids fort = 1 $\rightarrow$ négatif

$$B = -128 + 1 = \mathbf{-127}_{10}$$

- c) $C = 1111\ 1111_2 \rightarrow$ Bit de poids fort = 1 $\rightarrow$ négatif

$$C = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -128 + 127 = \mathbf{-1}_{10}$$

Correction : Décodage en Complément à 2

- a) $A = 0101\ 1010_2 \rightarrow$ Bit de poids fort = 0 $\rightarrow$ positif

$$A = 64 + 16 + 8 + 2 = \mathbf{90}_{10}$$

- b) $B = 1000\ 0001_2 \rightarrow$ Bit de poids fort = 1 $\rightarrow$ négatif

$$B = -128 + 1 = \mathbf{-127}_{10}$$

- c) $C = 1111\ 1111_2 \rightarrow$ Bit de poids fort = 1 $\rightarrow$ négatif

$$C = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -128 + 127 = \mathbf{-1}_{10}$$

- d) $D = 0111\ 1111_2 \rightarrow$ Bit de poids fort = 0 $\rightarrow$ positif

$$D = 64 + 32 + 16 + 8 + 4 + 2 + 1 = \mathbf{127}_{10}$$

(Valeur maximale sur 8 bits signés !)

Addition de nombres signés

Principe

L'addition fonctionne **exactement pareil** qu'avec des nombres non signés ! On additionne bit par bit avec retenues, puis on ignore la retenue finale.

Exemple 1 : $5 + (-3) = 2$ sur 8 bits.

Retenue	1	1	1	1	1	1		1	
+5		0	0	0	0	0	1	0	1
−3		1	1	1	1	1	1	0	1
Résultat	1	0	0	0	0	0	0	1	0

On ignore la retenue finale (en bleu) $\rightarrow 0000\ 0010_2 = +2$

Remarque importante

Le matériel n'a besoin que d'un seul additionneur pour signés ET non signés !

Débordement (overflow)

Problème

Que se passe-t-il si le résultat d'une opération dépasse la plage $[-128, +127]$?

Exemple : $100 + 50 = 150$ (overflow !)

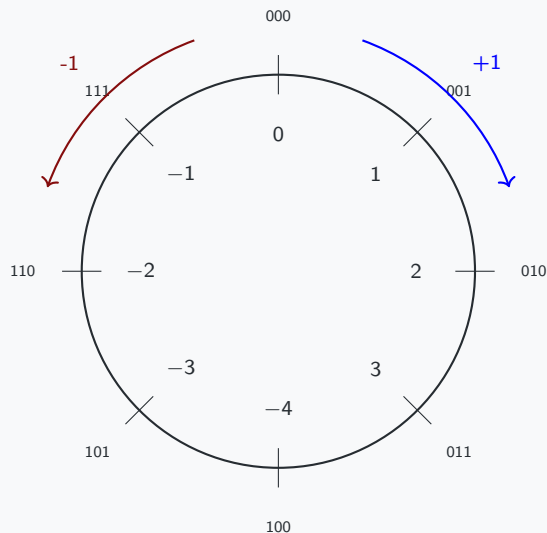
Retenue	0	1	1	1	0	0	0	0
+100	0	1	1	0	0	1	0	0
+50	0	0	1	1	0	0	1	0
Résultat	1	0	0	1	0	1	1	0

Résultat : $1001\ 0110_2 = -106$ en complément à 2 (absurde !)

Détection du débordement signé (flag V ou OV)

Overflow si et seulement si retenue entrante dans bit de poids fort $\neq$ retenue sortante. Ici : entrante = 0, sortante = 1 $\rightarrow V = 1$ (overflow détecté !)

Visualisation : La roue des nombres signés



Sur 3 bits

Plage : $[-4, +3]$

Exemples :

- $3 + 1 = 100_2 = -4$ (overflow)
- $-4 - 1 = 011_2 = +3$ (overflow)
- $-1 + 1 = 000_2 = 0$

Propriété

Le complément à 2 forme une **arithmétique modulaire** : les nombres "bouclent" !

Récapitulatif : Entiers signés

Complément à 2 (standard universel)

- Plage sur n bits : $[-2^{n-1}, 2^{n-1} - 1]$
- Bit de poids fort = bit de signe ($0 \rightarrow$ positif, $1 \rightarrow$ négatif)
- Formule de décodage : ${}^P(-N)_2 = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$
- Encodage de $-X$: inverser tous les bits de X puis ajouter 1

Avantages

Un seul zéro (contrairement à signe + valeur absolue)
 Addition/soustraction identiques aux non signés
 Matériel simplifié

Détection de débordement

- **Non signés** : flag C (carry) = retenue sortante
- **Signés** : flag V/OV (overflow) = retenue entrante $\neq$ retenue sortante

1. Notion de Codage
2. Entiers non signés
3. Entiers signés
4. Nombres à virgule flottante
5. Représentation des caractères
6. Exercices de synthèse

Nombres à virgule flottante

Problème avec les entiers

Les entiers ne peuvent représenter que des nombres entiers !

Comment coder 3.14159, 0.0001, 6.022×10^{23} ?

Deux approches :

1. **Virgule fixe** : nombre fixe de bits pour la partie entière et fractionnaire
 - Exemple : 8 bits entiers + 8 bits fractionnaires
 - Limité en plage et en précision

Nombres à virgule flottante

Problème avec les entiers

Les entiers ne peuvent représenter que des nombres entiers !

Comment coder 3.14159, 0.0001, 6.022×10^{23} ?

Deux approches :

1. **Virgule fixe** : nombre fixe de bits pour la partie entière et fractionnaire
 - Exemple : 8 bits entiers + 8 bits fractionnaires
 - Limité en plage et en précision
2. **Virgule flottante** (standard IEEE 754) : notation scientifique en binaire
 - Signe + Mantisse + Exposant
 - Très grande plage dynamique
 - **Standard universel** sur tous les ordinateurs modernes

Attention

Les flottants ont des **limitations de précision** !

Virgule fixe (Fixed-Point)

Principe

On réserve un nombre fixe de bits pour la partie entière et la partie fractionnaire.

Exemple sur 16 bits : 8.8 (8 bits entiers, 8 bits fractionnaires)

Partie entière (8 bits)	Partie fractionnaire (8 bits)
$2^7 \dots 2^1 2^0$	$2^{-1} 2^{-2} \dots 2^{-7} 2^{-8}$

Exemple : 5.75_{10}

Virgule fixe (Fixed-Point)

Principe

On réserve un nombre fixe de bits pour la partie entière et la partie fractionnaire.

Exemple sur 16 bits : 8.8 (8 bits entiers, 8 bits fractionnaires)

Partie entière (8 bits)	Partie fractionnaire (8 bits)
$2^7 \dots 2^1 2^0$	$2^{-1} 2^{-2} \dots 2^{-7} 2^{-8}$

Exemple : 5.75_{10}

- Partie entière : $5 = 0000\ 0101_2$

Virgule fixe (Fixed-Point)

Principe

On réserve un nombre fixe de bits pour la partie entière et la partie fractionnaire.

Exemple sur 16 bits : 8.8 (8 bits entiers, 8 bits fractionnaires)

Partie entière (8 bits)	Partie fractionnaire (8 bits)
$2^7 \dots 2^1 2^0$	$2^{-1} 2^{-2} \dots 2^{-7} 2^{-8}$

Exemple : 5.75_{10}

- Partie entière : $5 = 0000\ 0101_2$
- Partie fractionnaire : $0.75 = 0.5 + 0.25 = 2^{-1} + 2^{-2} = 1100\ 0000_2$

Virgule fixe (Fixed-Point)

Principe

On réserve un nombre fixe de bits pour la partie entière et la partie fractionnaire.

Exemple sur 16 bits : 8.8 (8 bits entiers, 8 bits fractionnaires)

Partie entière (8 bits)	Partie fractionnaire (8 bits)
$2^7 \dots 2^1 2^0$	$2^{-1} 2^{-2} \dots 2^{-7} 2^{-8}$

Exemple : 5.75_{10}

- Partie entière : $5 = 0000\ 0101_2$
- Partie fractionnaire : $0.75 = 0.5 + 0.25 = 2^{-1} + 2^{-2} = 1100\ 0000_2$
- Résultat : $0000\ 0101 . 1100\ 0000$

Limites

Plage restreinte (sur 8.8 : de 0 à 255.996)

Pas de nombres très grands ou très petits (10^{23} ou 10^{-10})

IEEE 754 : Norme universelle

Principe

Représentation en **notation scientifique binaire** :

$$\text{Nombre} = (-1)^S \times (1.M) \times 2^{E-\text{biais}}$$

Format simple précision (32 bits) :



- **S** (1 bit) : signe (0 = positif, 1 = négatif)
- **E** (8 bits) : exposant biaisé (biais = 127)
- **M** (23 bits) : mantisse (partie fractionnaire, bit implicite = 1)

Exemple : Décodage IEEE 754

Soit le nombre codé sur 32 bits :

0 10000010 010000000000000000000000

Décomposition :

- $S = 0 \rightarrow$ nombre positif
- $E = 10000010_2 = 130_{10}$
- $M = 010000000000000000000000_2 = 0.25_{10}$ (en binaire : $0.01_2 = 2^{-2}$)

Calcul :

$$\begin{aligned}\text{Valeur} &= (-1)^0 \times (1 + 0.25) \times 2^{(130-127)} \\ &= 1 \times 1.25 \times 2^3 \\ &= 1.25 \times 8 \\ &= \mathbf{10.0}\end{aligned}$$

Exercice : Décodage IEEE 754

Consigne

Décodez le nombre IEEE 754 suivant (32 bits) :

1 01111100 100000000000000000000000

Étapes :

1. Extraire S, E, M
2. Convertir E en décimal
3. Convertir M en décimal
4. Appliquer la formule : $(-1)^S \times (1.M) \times 2^{(E-127)}$

Correction : Décodage IEEE 754

1 01111100 100000000000000000000000

Décomposition :

- $S = 1 \rightarrow$ nombre **négalif**
- $E = 01111100_2 = 124_{10}$
- $M = 100000000000000000000000_2 = 0.5_{10}$
(en binaire : $0.1_2 = 2^{-1} = 0.5$)

Calcul :

$$\begin{aligned}
 \text{Valeur} &= (-1)^1 \times (1 + 0.5) \times 2^{(124-127)} \\
 &= -1 \times 1.5 \times 2^{-3} \\
 &= -1.5 \times \frac{1}{8} \\
 &= -1.5 \times 0.125 \\
 &= \mathbf{-0.1875}
 \end{aligned}$$

Codage d'un nombre en IEEE 754

Étapes pour coder un nombre (ex : -6.5) :

1. **Signe** : -6.5 est négatif $\rightarrow S = 1$

Codage d'un nombre en IEEE 754

Étapes pour coder un nombre (ex : -6.5) :

1. **Signe** : -6.5 est négatif $\rightarrow S = 1$
2. **Conversion en binaire** : $6.5 = 110.1_2$
(car $6 = 110_2$ et $0.5 = 0.1_2$)

Codage d'un nombre en IEEE 754

Étapes pour coder un nombre (ex : -6.5) :

1. **Signe** : -6.5 est négatif $\rightarrow S = 1$
2. **Conversion en binaire** : $6.5 = 110.1_2$
(car $6 = 110_2$ et $0.5 = 0.1_2$)
3. **Normalisation** : écrire sous forme $1.M \times 2^{\text{exp}}$
 $110.1_2 = 1.101_2 \times 2^2$

Codage d'un nombre en IEEE 754

Étapes pour coder un nombre (ex : -6.5) :

1. **Signe** : -6.5 est négatif $\rightarrow S = 1$
2. **Conversion en binaire** : $6.5 = 110.1_2$
(car $6 = 110_2$ et $0.5 = 0.1_2$)
3. **Normalisation** : écrire sous forme $1.M \times 2^{\text{exp}}$
 $110.1_2 = 1.101_2 \times 2^2$
4. **Exposant biaisé** : $E = 2 + 127 = 129 = 10000001_2$

Codage d'un nombre en IEEE 754

Étapes pour coder un nombre (ex : -6.5) :

1. **Signe** : -6.5 est négatif $\rightarrow S = 1$
2. **Conversion en binaire** : $6.5 = 110.1_2$
(car $6 = 110_2$ et $0.5 = 0.1_2$)
3. **Normalisation** : écrire sous forme $1.M \times 2^{\text{exp}}$
 $110.1_2 = 1.101_2 \times 2^2$
4. **Exposant biaisé** : $E = 2 + 127 = 129 = 10000001_2$
5. **Mantisse** : $M =$ partie après "1." $\rightarrow$ 101 complété à 23 bits :
10100000000000000000000

Résultat final :

1 10000001 10100000000000000000000

Valeurs spéciales IEEE 754

Valeur	S	E (8 bits)	M (23 bits)
+0	0	00000000	00000000000000000000000
−0	1	00000000	00000000000000000000000
$+\infty$	0	11111111	00000000000000000000000
$-\infty$	1	11111111	00000000000000000000000
NaN (Not a Number)	X	11111111	$\neq 0$ (quelconque)

Exemples d'opérations produisant NaN

- $\frac{0}{0}$, $\infty - \infty$, $\sqrt{-1}$
- NaN se propage : toute opération avec NaN donne NaN

Attention

IEEE 754 distingue +0 et −0 (mais sont égaux en comparaison)

Pièges des nombres flottants

Piège 1 : Précision limitée (arrondi)

De nombreux nombres décimaux n'ont PAS de représentation binaire exacte !

Exemple classique : $0.1 + 0.2 \neq 0.3$ en flottant !

```
>>> 0.1 + 0.2  
0.30000000000000004
```

Raison : $0.1_{10} = 0.0001100110011\dots_2$ (période infinie en binaire !)

Règle d'or

Ne JAMAIS comparer deux flottants avec `==`

À la place : $|a - b| < \varepsilon$ (avec ε très petit, ex : 10^{-9})

1. Notion de Codage
2. Entiers non signés
3. Entiers signés
4. Nombres à virgule flottante
- 5. Représentation des caractères**
6. Exercices de synthèse

Représentation des caractères

Problème

Comment représenter les lettres, chiffres, symboles sous forme binaire ?

Historique des encodages :

1. **ASCII (1963)** : 7 bits, 128 caractères (alphabet anglais)
 - A-Z, a-z, 0-9, ponctuation, caractères de contrôle

Représentation des caractères

Problème

Comment représenter les lettres, chiffres, symboles sous forme binaire ?

Historique des encodages :

1. **ASCII (1963)** : 7 bits, 128 caractères (alphabet anglais)
 - A-Z, a-z, 0-9, ponctuation, caractères de contrôle
2. **Variantes nationales (années 70-80)** : ISO-8859-1 (Latin-1), etc.
 - 8 bits, 256 caractères (ajout des accents européens)
 - Incompatibilités entre pays !

Représentation des caractères

Problème

Comment représenter les lettres, chiffres, symboles sous forme binaire ?

Historique des encodages :

1. **ASCII (1963)** : 7 bits, 128 caractères (alphabet anglais)
 - A-Z, a-z, 0-9, ponctuation, caractères de contrôle
2. **Variantes nationales (années 70-80)** : ISO-8859-1 (Latin-1), etc.
 - 8 bits, 256 caractères (ajout des accents européens)
 - Incompatibilités entre pays !
3. **Unicode (1991)** : norme universelle pour TOUS les caractères
 - Plus de 149 000 caractères (toutes les langues, émojis, etc.)
 - Encodages : UTF-8, UTF-16, UTF-32

Table ASCII (7 bits, 128 caractères)

Dec	Hex	Bin	Char	Dec	Hex	Bin	Char
32	20	0100000	(espace)	65	41	1000001	A
33	21	0100001	!	66	42	1000010	B
...	...	...	...	...	...	...	...
48	30	0110000	0	90	5A	1011010	Z
49	31	0110001	1	97	61	1100001	a
...	...	...	...	98	62	1100010	b
57	39	0111001	9	...	...	...	...

Points clés

- **A-Z** : codes 65 à 90 (majuscules)
- **a-z** : codes 97 à 122 (minuscules, décalage de +32)
- **0-9** : codes 48 à 57 (chiffres)
- Codes 0-31 : caractères de contrôle (retour chariot, tabulation, etc.)

Exercice : Décodage ASCII

Consigne

Décodez la séquence hexadécimale suivante en texte ASCII :

49 4E 46 4F 35 30 31

Méthode : Convertir chaque octet en décimal, puis chercher dans la table ASCII.

Exercice : Décodage ASCII

Consigne

Décodez la séquence hexadécimale suivante en texte ASCII :

49 4E 46 4F 35 30 31

Méthode : Convertir chaque octet en décimal, puis chercher dans la table ASCII.

Correction

- $49_{16} = 73_{10} \rightarrow \text{'I'}$
- $4E_{16} = 78_{10} \rightarrow \text{'N'}$
- $46_{16} = 70_{10} \rightarrow \text{'F'}$
- $4F_{16} = 79_{10} \rightarrow \text{'O'}$
- $35_{16} = 53_{10} \rightarrow \text{'5'}$
- $30_{16} = 48_{10} \rightarrow \text{'0'}$
- $31_{16} = 49_{10} \rightarrow \text{'1'}$

Limites de l'ASCII

Problèmes de l'ASCII

- **Limité à 128 caractères** (7 bits) → pas d'accents !
- Extensions nationales (ISO-8859-1, Windows-1252, etc.) **incompatibles** entre elles
- Impossible de représenter les langues non-latines (chinois, arabe, cyrillique, etc.)
- Pas d'émojis, symboles mathématiques avancés, etc.

Exemple de problème :

- Un fichier avec des "é" codé en Latin-1 lu en UTF-8 → "Ã©"
- Un email en cyrillique lu avec l'encodage japonais → illisible !

Solution : Unicode

Norme universelle assignant un **point de code** unique à chaque caractère de toutes les langues du monde !

Unicode et UTF-8 I

Unicode

- Norme définissant plus de 149 000 caractères
- Chaque caractère a un **point de code** noté U+XXXX (en hexa)
- Exemples : U+0041 = 'A', U+00E9 = 'é', U+1F600 = :) (à imaginer)

UTF-8 : encodage dominant (web, Linux, etc.)

Encodage à longueur variable : 1 à 4 octets par caractère

- **1 octet** : caractères ASCII (0-127) → compatibilité totale !
- **2 octets** : accents, cyrillique, arabe, hébreu, etc.
- **3 octets** : chinois, japonais, coréen
- **4 octets** : émojis, caractères rares

Unicode et UTF-8 II

Avantage de UTF-8

Rétrocompatible avec ASCII
Économie de l'espace pour les textes latins
Standard du web (>98% des sites)

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxxyy	10yyzzzz

Exemple : Encodage UTF-8

Caractère : 'é' (e accent aigu)

- Point de code Unicode : U+00E9 (décimal : 233)
- Encodage UTF-8 : 2 octets : C3 A9

Décomposition :

Octet 1	Octet 2
11000011 (C3)	10101001 (A9)

- Premier octet commence par 110 → caractère sur 2 octets
- Deuxième octet commence par 10 → octet de continuation
- Bits de données : 00011 101001 = 233_{10}

Émoji : :) (U+1F600)

- Encodage UTF-8 : 4 octets : F0 9F 98 80

Art ASCII (pour le plaisir !)



Chaque caractère est codé en ASCII !
 Art créé uniquement avec des caractères imprimables.

Anecdote

L'art ASCII était populaire dans les années 80-90 avant l'arrivée des interfaces graphiques. On le retrouve encore dans les README, bannières de logiciels CLI, etc.

Récapitulatif : Caractères I

ASCII (1963)

- 7 bits, 128 caractères (alphabet anglais + contrôle)
- A = 65, a = 97, 0 = 48
- Base de tous les encodages modernes

Unicode (1991)

- Norme universelle : >149 000 caractères (toutes langues + émojis)
- Point de code : U+XXXX (notation hexadécimale)

Récapitulatif : Caractères II

UTF-8 (encodage dominant)

- Longueur variable : 1 à 4 octets
- Rétrocompatible avec ASCII (1 octet pour 0-127)
- Standard du web (>98%)

Attention

Toujours spécifier l'encodage lors de la lecture/écriture de fichiers texte !

1. Notion de Codage
2. Entiers non signés
3. Entiers signés
4. Nombres à virgule flottante
5. Représentation des caractères
6. Exercices de synthèse

Exercice 1 : Transstockeur (version simplifiée)

Contexte

Un transstockeur industriel utilise des adresses codées sur 12 bits :

- 4 bits pour l'allée (A)
- 4 bits pour la travée (T)
- 4 bits pour le niveau (N)

Format : AAAA TTTT NNNN

Questions :

1. Combien d'emplacements différents peut-on adresser ?
2. Quelle est l'adresse (en binaire et en hexa) de l'emplacement :
Allée 5, Travée 12, Niveau 3 ?
3. Décodez l'adresse hexadécimale 0x7A9.

Correction : Transstockeur

1) Nombre d'emplacements :

$$2^{12} = 4096 \text{ emplacements}$$

$$(\text{ou } 2^4 \times 2^4 \times 2^4 = 16 \times 16 \times 16 = 4096)$$

2) Encodage de (Allée 5, Travée 12, Niveau 3) :

- Allée 5 : $5 = 0101_2$
- Travée 12 : $12 = 1100_2$
- Niveau 3 : $3 = 0011_2$

Adresse binaire : 0101 1100 0011₂

Adresse hexa : 0x5C3

3) Décodage de 0x7A9 :

$$0x7A9 = 0111\ 1010\ 1001_2$$

- Allée : $0111_2 = 7$
- Travée : $1010_2 = 10$
- Niveau : $1001_2 = 9$

Résultat : Allée 7, Travée 10, Niveau 9

Exercice 2 : Numération et arithmétique

a) Conversions :

1. Convertir 237_{10} en binaire, octal, et hexadécimal
2. Convertir 10110011_2 en décimal et hexadécimal

b) Addition binaire sur 8 bits (non signés) :

Calculer $01101101_2 + 00111010_2$. Y a-t-il débordement (carry) ?

c) Complément à 2 sur 8 bits :

1. Encoder -42 en complément à 2
2. Décoder 11010110_2 (complément à 2)

Correction : Numération et arithmétique

a) Conversions de 237_{10} :

- Binaire : $237 = 128 + 64 + 32 + 8 + 4 + 1 = 11101101_2$
- Octal : $237 \div 8 = 29$ reste 5, $29 \div 8 = 3$ reste 5, $3 \div 8 = 0$ reste 3 $\rightarrow 355_8$
- Hexa : $11101101_2 = 1110 \ 1101 = ED_{16}$

Conversions de 10110011_2 :

- Décimal : $128 + 32 + 16 + 2 + 1 = 179_{10}$
- Hexa : $1011 \ 0011 = B3_{16}$

b) Addition : $01101101_2 + 00111010_2$

$$01101101 + 00111010 = 10100111_2 \quad (\text{pas de retenue sortante}) \rightarrow \text{Carry} = 0$$

Résultat : $10100111_2 = 167_{10}$ (pas de débordement)

Correction : Complément à 2

c.1) Encoder -42 en complément à 2 (8 bits) :

1. Encoder $+42$: $42 = 00101010_2$
2. Inverser tous les bits : 11010101_2
3. Ajouter 1 : $11010101 + 1 = 11010110_2$

Résultat : $-42 = 11010110_2$

c.2) Décoder 11010110_2 (complément à 2) :

Bit de poids fort = 1 $\rightarrow$ négatif

$$-128 + 64 + 16 + 4 + 2 = -128 + 86 = -42_{10}$$

Cohérent avec l'encodage précédent !

Exercice 3 : IEEE 754 (optionnel / avancé)

Question

Encoder le nombre -12.5 en IEEE 754 simple précision (32 bits).

Étapes suggérées :

1. Déterminer le signe (S)
2. Convertir 12.5 en binaire
3. Normaliser sous forme $1.M \times 2^{\text{exp}}$
4. Calculer l'exposant biaisé (E)
5. Extraire la mantisse (M) sur 23 bits
6. Assembler : S | E (8 bits) | M (23 bits)

Exercice 3 : IEEE 754 (optionnel / avancé)

Question

Encoder le nombre -12.5 en IEEE 754 simple précision (32 bits).

Étapes suggérées :

1. Déterminer le signe (S)
2. Convertir 12.5 en binaire
3. Normaliser sous forme $1.M \times 2^{\text{exp}}$
4. Calculer l'exposant biaisé (E)
5. Extraire la mantisse (M) sur 23 bits
6. Assembler : S | E (8 bits) | M (23 bits)

Indice

$$12.5_{10} = 12 + 0.5 = 1100_2 + 0.1_2 = 1100.1_2$$

Correction : IEEE 754 (-12.5)

1) **Signe** : -12.5 est négatif $\rightarrow S = 1$

2) **Conversion binaire** : $12.5 = 1100.1_2$
(car $12 = 1100_2$ et $0.5 = 0.1_2$)

3) **Normalisation** : $1100.1_2 = 1.1001_2 \times 2^3$

4) **Exposant biaisé** : $E = 3 + 127 = 130 = 10000010_2$

5) **Mantisse** : $M =$ partie après "1." $\rightarrow$ 1001 complété à 23 bits :
10010000000000000000000

Résultat final :

1 10000010 10010000000000000000000

En hexadécimal : 0xC1480000

Conclusion

Ce que nous avons vu aujourd'hui

- **Notion de codage** : tout est binaire en machine
- **Entiers non signés** : conversions, bases 2/8/16, arithmétique
- **Entiers signés** : complément à 2, débordement
- **Nombres flottants** : IEEE 754, pièges, bugs célèbres
- **Caractères** : ASCII, Unicode, UTF-8

Points clés à retenir

- Les nombres ont une **plage limitée** (overflow possibles)
- Les flottants ont une **précision limitée** (ne jamais comparer avec ==)
- Les caractères dépendent de l'**encodage** (toujours spécifier UTF-8 !)

BONUS

Manipulation de la mémoire en hexadécimal

Ce que nous allons voir

- Lancer un mini-jeu qui affiche ses variables en mémoire
- Affichage en temps réel : hexadécimal
- Modifier la mémoire avec un débogueur
- Observer le changement instantané des valeurs

Passons à la démo...